HubSpot Integration

1. Overview

1.1 Description

HubSpot Integration connects **HubSpot** (CRM and marketing platform) with **Newo Platform**.

It enables:

- Checking real-time appointment availability via HubSpot meeting links
- Booking appointments with automatic contact creation in HubSpot CRM
- Cancelling existing appointments (meeting deletion)
- Existing customer detection with upcoming booking retrieval
- Automatic customer context injection into agent prompt
- OAuth 2.0 authentication with automatic token refresh

This integration is designed for **sales teams, service businesses, and professionals** who need **automated appointment scheduling and contact management through a conversational AI agent** (voice or chat).

1.2 Use Cases

- When a client asks about available slots → Query HubSpot meeting link scheduler and return free time slots
- When a client wants to book an appointment → Find or create contact, then book meeting via HubSpot scheduler
- When a client wants to cancel an appointment → Find and delete meeting from HubSpot CRM
- When a caller's phone is identified → Look up existing contact and inject their context + upcoming bookings into prompt
- When a conversation starts → Optionally preload nearest available slots silently

1.3 Supported Features

Feature	Supported	Notes
Check Availability	✅	Via HubSpot meeting link scheduler API
Book Appointment	✅	Creates meeting via scheduler with auto contact creation
Cancel Appointment	✅	Finds meeting by start time, then deletes from CRM
Existing Client Detection	✅	By phone/email, with prompt context injection
Upcoming Booking Retrieval	✅	Future meetings stored in persona
Silent Availability Preload	✅	Optional check on conversation start
Configurable Slot Duration	✅	Default 30 min, overridable per request
Configurable Booking Window	✅	Days ahead to show (default: 1)
Timezone Handling	✅	Converts between local and UTC

Reschedule Appointment	❌	Cancel + rebook as workaround
Deal/Pipeline Management	❌	Not implemented
Multi-Calendar Support	❌	Single meeting link (slug) per integration

2. Requirements

Before installation:

- Active **HubSpot** account with Meeting Scheduler enabled
- **HubSpot OAuth App** with Client ID and Client Secret
- A **Meeting Link slug** (the URL path of your meeting link, e.g., `john-doe`)
- HubSpot API base URL (default: `https://api.hubapi.com`)

3. How to Setup

3.1 Step 1 — Authorize in HubSpot

1. In Newo platform, set `hubspot_client_id`, `hubspot_client_secret`, and `hubspot_redirect_url`
2. Click the HubSpot authorization link provided by the platform
3. Log in to your HubSpot account
4. Authorize the Newo app and grant requested scopes
5. Copy the **authorization code** from the redirect URL
6. Paste the code into the `hubspot_oauth_code` attribute

3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

Attribute	Required	Description
`hubspot_client_id`	✅	OAuth App ID
`hubspot_client_secret`	✅	OAuth App Secret
`hubspot_redirect_url`	✅	OAuth redirect URL
`hubspot_oauth_code`	✅	One-time OAuth authorization code
`hubspot_slug_name`	✅	Meeting link URL slug (e.g., `john-doe`)
`hubspot_base_api_url`	❌	API base URL (default: `https://api.hubapi.com`)
`hubspot_timezone`	❌	Appointment timezone (default: project timezone)
`hubspot_duration`	❌	Slot duration in minutes (default: `30`)
`hubspot_show_for_days`	❌	Days ahead for availability (default: `1`)
`hubspot_slots_available`	❌	Enable availability checking (default: `True`)
`hubspot_booking_available`	❌	Enable booking capability (default: `True`)

`hubspot_cancelation_available`	❌	Enable cancellation capability (default: `True`)

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Exchanges the OAuth code for access and refresh tokens
    - Validates token persistence via test API call
    - Creates HTTP connectors for HubSpot API
    - Injects booking and availability payload schemas for the agent

4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

4.1 How the Integration Works

The integration uses HubSpot **Meeting Links** for scheduling and **Contacts** for customer management. Availability is computed from the meeting link's busy times, and bookings are created via the scheduler API.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in HubSpot

Available Triggers

Trigger	Event	Description
Check Availability	`get_available_slots`	When the agent needs to look up open slots
Book Appointment	`book_slot`	When the agent confirms a booking
Cancel Appointment	`convo_agent_cancel_booking`	When the agent processes a cancellation
Phone Identified	`define_user_phone_number`	When the caller's phone is detected
Session Start	`session_started`	Optional silent availability preload
Setup	`project_publish_finish`	Initializes integration on deploy

Available Actions

- **Search Contact** — Find contact by email or phone (POST `/crm/v3/objects/contacts/search`)
- **Create Contact** — New contact with name, phone, email (POST `/crm/v3/objects/contacts`)
- **Get Availability** — Fetch busy times from meeting link (GET `/scheduler/v3/meetings/meeting-links/book/{slug}`)
- **Book Meeting** — Schedule appointment via meeting link (POST `/scheduler/v3/meetings/meeting-links/book`)
- **Search Meetings** — Find meetings by filters (POST `/crm/v3/objects/meetings/search`)
- **Delete Meeting** — Cancel appointment (DELETE `/crm/v3/objects/meetings/{id}`)

4.2 Architecture Diagrams

Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
```

```
    participant I as HubSpot Integration
    participant T as HubSpot OAuth
    participant API as HubSpot API

    Note over I,T: Setup (on publish)
    A->>I: project_publish_finish
    I->>T: POST /oauth/v1/token (code)
    T-->>I: access_token + refresh_token
    I->>API: GET /crm/v3/objects/contacts?limit=1
    API-->>I: Token verified
    I->>A: Schemas injected, tools configured

    Note over U,API: Existing Client Detection
    A->>I: define_user_phone_number (phone)
    I->>API: POST /crm/v3/objects/contacts/search (phone, email)
    API-->>I: Contact data
    I->>A: Inject ExistingCustomerInfo into prompt
    I->>API: POST /crm/v3/objects/meetings/search (contact_id)
    API-->>I: Future meetings
    I->>A: Store bookings in persona

    Note over U,API: Availability Check
    U->>A: "Any slots tomorrow?"
    A->>I: get_available_slots (date, time)
    I->>API: GET /scheduler/v3/meetings/meeting-links/book/(slug)?timezone=(tz)
    API-->>I: Busy times
    I->>I: Calculate free slots (subtract busy, slice by duration)
    I->>A: result_success (availability[])
    A->>U: "Here are the available times..."

    Note over U,API: Booking
    U->>A: "Book 2:00 PM please"
    A->>I: book_slot (customerInfo, dateTime)
    I->>API: POST /crm/v3/objects/contacts/search (email)
    alt Contact not found
        I->>API: POST /crm/v3/objects/contacts (name, phone, email)
        API-->>I: contact_id
    end
    I->>API: POST /scheduler/v3/meetings/meeting-links/book (slug, startTime,
duration, contact)
    API-->>I: calendarEventId
    I->>A: result_success (booking_id)
    A->>U: "Your appointment is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my appointment"
    A->>I: convo_agent_cancel_booking (booking_id, dateTime)
    I->>API: POST /crm/v3/objects/meetings/search (start_time)
    API-->>I: meeting_id
    I->>API: DELETE /crm/v3/objects/meetings/(meeting_id)
    API-->>I: Confirmation
```

```
    I->>A: result_success (cancelled)
    A->>U: "Your appointment has been cancelled"
```

AvailabilityFlow

```
flowchart TD
    E["get_available_slots<br>or session_started"] --> GATE{{"Slot
check<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| RANGE["Calculate UTC window:<br>date → date + show_for_days"]
    RANGE --> API["GET /scheduler/v3<br>/meetings/meeting-links<br>/book/{slug}<br>?
timezone={tz}"]
    API --> OK{{"HTTP 200?"}}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| PARSE["Parse allUsersBusyTimes<br>(ms timestamps)"]
    PARSE --> CALC["Generate slots at<br>duration intervals<br>Subtract busy times"]
    CALC --> TZ["Convert to<br>local timezone<br>(HH:MM AM/PM)"]
    TZ --> OUT["result_success<br>{availability[]}"]
```

BookingFlow

```
flowchart TD
    E["book_slot"] --> GATE{{"Booking<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| SEARCH["POST /crm/v3/objects<br>/contacts/search<br>{email}"]
    SEARCH --> FOUND{{"Contact<br>found?"}}
    FOUND -->|"Yes"| PID["Use contact_id"]
    FOUND -->|"No"| CREATE["POST /crm/v3/objects<br>/contacts<br>{name, phone,
email}"]
    CREATE --> PID
    PID --> UTC["Convert time<br>to milliseconds (UTC)"]
    UTC --> API["POST /scheduler/v3<br>/meetings/meeting-links/book<br>{slug,
firstName, lastName,<br>email, startTime, duration,<br>timezone}"]
    API --> OK{{"HTTP 200?"}}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| OUT["result_success<br>{booking_id:<br>calendarEventId}"]
```

CancellationFlow

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{{"Cancellation<br>enabled?"}}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| UTC["Convert datetime<br>to UTC"]
    UTC --> FIND["POST /crm/v3/objects<br>/meetings/search<br>{start_time EQ
filter}"]
    FIND --> FOUND{{"Meeting<br>found?"}}
    FOUND -->|"No"| ERR["ResultError"]
    FOUND -->|"Yes"| DEL["DELETE /crm/v3<br>/objects/meetings<br>/{meeting_id}"]
    DEL --> OK{{"Success?"}}
```

```
    OK -->|"No"| ERR
    OK -->|"Yes"| OUT["result_success<br>{cancel_booking}"]
```

ExistingClientFlow

```
flowchart TD
    E["define_user_phone_number"] --> SEARCH["POST
/crm/v3/objects<br>/contacts/search<br>{phone OR email}"]
    SEARCH --> FOUND{{"Contact<br>found?"}}
    FOUND -->|"No"| DONE["No action"]
    FOUND -->|"Yes"| INJECT["Inject ExistingCustomerInfo<br>into agent prompt<br>
{name, email, phone}"]
    INJECT --> APPTS["POST /crm/v3/objects<br>/meetings/search<br>{contact_id
association}"]
    APPTS --> FILTER["Filter future<br>meetings only"]
    FILTER --> SAVE["Store in persona<br>attribute: bookings<br>(deduplicated)"]
```

UtilsFlow (Token Refresh)

```
flowchart TD
    REQ["Any API request"] --> HTTP["Send via<br>hubspot_connector"]
    HTTP --> STATUS{{"HTTP<br>status?"}}
    STATUS -->|"200"| CHECK{{"Response has<br>error field?"}}
    CHECK -->|"No"| SUCCESS["hubspot_result_success"]
    CHECK -->|"Yes"| ERR["hubspot_result_error"]
    STATUS -->|"401"| REFRESH["POST /oauth/v1/token<br>{grant_type:
<br>refresh_token}"]
    REFRESH --> TOK_OK{{"New token<br>received?"}}
    TOK_OK -->|"Yes"| SAVE["Save tokens<br>Retry request<br>(max 3 attempts)"]
    TOK_OK -->|"No"| ERR
    STATUS -->|"4xx/5xx"| ERR
    SAVE --> HTTP
```

4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel). BookingFlow has a dedicated webhook test endpoint (`book_wh`) for connectivity validation.

4.4 Testing and Verification

To test the integration:

1. **Setup**: Publish the project and verify OAuth token exchange succeeds
2. **Existing Client**: Start a conversation from a known phone — verify customer info injected into prompt
3. **Availability**: Ask "What slots are available tomorrow?" — verify free slots calculated from meeting link schedule
4. **Booking**: Complete a booking — verify meeting created in HubSpot with correct contact link
5. **Cancellation**: Cancel a booking — verify meeting deleted from HubSpot CRM

If no action occurs:

- Ensure OAuth credentials are set (`hubspot_client_id` , `hubspot_client_secret`)
- Verify `hubspot_access_token` was obtained (check it's not empty after setup)
- Confirm `hubspot_slug_name` matches an existing meeting link in HubSpot
- Check feature flags are enabled (`hubspot_slots_available` , `hubspot_booking_available` , `hubspot_cancelation_available`)
- Verify timezone is set correctly via `hubspot_timezone`
- Re-publish the project if credentials were changed

> *Note: This integration performs real operations in HubSpot. Contacts and meetings created or deleted through the agent are reflected in the live CRM system.*

5. FAQ

Q: Does the integration import historical data? A: No. Only meetings created after activation are managed. Existing client detection retrieves future meetings for context only.

Q: Can I connect multiple HubSpot accounts? A: Each integration instance supports one HubSpot account and meeting link. For multiple accounts, create separate integration instances.

Q: How does availability checking work? A: The integration queries the HubSpot meeting link's `allUsersBusyTimes` and generates free time slots by finding gaps. Slot duration is configurable via `hubspot_duration` (default: 30 minutes).

Q: What is the meeting link slug? A: The slug is the URL path of your HubSpot meeting link (e.g., `john-doe` from `meetings.hubspot.com/john-doe`). Set it in `hubspot_slug_name` — it determines which calendar is used for availability and booking.

Q: What happens if a contact already exists? A: The integration searches contacts by email. If found, the existing contact is used for booking. If not, a new contact is created with name, phone, and email.

Q: How does cancellation find the right meeting? A: Cancellation searches HubSpot meetings by exact start time match (EQ filter on `hs_meeting_start_time`). The meeting ID from the search result is then used for deletion.

Q: What happens if the OAuth token expires? A: The integration automatically detects HTTP 401 responses and refreshes the token using the refresh token. It retries the failed request up to 3 times. A separate HTTP connector is used for token operations to ensure isolation.

Q: Can the slot duration vary per booking? A: Yes. While the default is set via `hubspot_duration` , each booking request can include a duration override in the payload. This allows flexible appointment lengths within the same integration instance.